

Assignment5

Info

Name: Cody (Zhexian) Liu

Assignment Number: 5

Repo Link: <https://github.com/Rorical/CWM-FDI>

Resolving URLs using DNS

1. Resolve addresses of 3 major websites

```
ubuntu@ubuntu:~$ nslookup www.google.com
```

```
Server:      127.0.0.53
```

```
Address: 127.0.0.53#53
```

```
Non-authoritative answer:
```

```
Name:      www.google.com
```

```
Address: 142.251.153.119
```

```
Name:      www.google.com
```

```
Address: 142.251.152.119
```

```
Name:      www.google.com
```

```
Address: 142.251.154.119
```

```
Name:      www.google.com
```

```
Address: 142.251.156.119
```

```
Name:      www.google.com
```

```
Address: 142.251.151.119
```

```
Name:      www.google.com
```

```
Address: 142.251.155.119
```

```
Name:      www.google.com
```

```
Address: 142.251.157.119
```

```
Name:      www.google.com
```

```
Address: 142.251.150.119
```

```
Name:      www.google.com
```

```
Address: 2001:4860:4829:7700::
```

```
Name:      www.google.com
```

```
Address: 2001:4860:4826:7700::
```

```
Name:      www.google.com
```

```
Address: 2001:4860:482b:7700::
```

```
Name:      www.google.com
```

```
Address: 2001:4860:4828:7700::
```

```
Name:      www.google.com
```

```
Address: 2001:4860:482a:7700::
```

```
Name:      www.google.com
```

Address: 2001:4860:4827:7700::
Name: www.google.com
Address: 2001:4860:482d:7700::
Name: www.google.com
Address: 2001:4860:482c:7700::

ubuntu@ubuntu:~\$ nslookup www.amazon.com
Server: 127.0.0.53
Address: 127.0.0.53#53

Non-authoritative answer:

www.amazon.com canonical name = tp.47cf2c8c9-frontier.amazon.com.
tp.47cf2c8c9-frontier.amazon.com canonical name = cf.47cf2c8c9-frontier.amazon.com.

Name: cf.47cf2c8c9-frontier.amazon.com
Address: 99.86.89.127
Name: cf.47cf2c8c9-frontier.amazon.com
Address: 2600:9000:2117:8800:7:49a5:5fd6:da1
Name: cf.47cf2c8c9-frontier.amazon.com
Address: 2600:9000:2117:2a00:7:49a5:5fd6:da1
Name: cf.47cf2c8c9-frontier.amazon.com
Address: 2600:9000:2117:c600:7:49a5:5fd6:da1
Name: cf.47cf2c8c9-frontier.amazon.com
Address: 2600:9000:2117:d400:7:49a5:5fd6:da1
Name: cf.47cf2c8c9-frontier.amazon.com
Address: 2600:9000:2117:400:7:49a5:5fd6:da1
Name: cf.47cf2c8c9-frontier.amazon.com
Address: 2600:9000:2117:3200:7:49a5:5fd6:da1
Name: cf.47cf2c8c9-frontier.amazon.com
Address: 2600:9000:2117:ac00:7:49a5:5fd6:da1
Name: cf.47cf2c8c9-frontier.amazon.com
Address: 2600:9000:2117:7e00:7:49a5:5fd6:da1

ubuntu@ubuntu:~\$ nslookup www.wikipedia.org
Server: 127.0.0.53
Address: 127.0.0.53#53

Non-authoritative answer:

www.wikipedia.org canonical name = dyna.wikimedia.org.
Name: dyna.wikimedia.org
Address: 185.15.59.224
Name: dyna.wikimedia.org
Address: 2a02:ec80:300:ed1a::1

Website	CNAME chain	IPv4 addresses	IPv6 addresses	Total
www.google.com	No	8	8	16
www.amazon.com	Yes (2 hops)	1	8	9
www.wikipedia.org	Yes (1 hop)	1	1	2

Google returns many IP address results, which indicate that it has a strong load-balancing and CDN for requests, and these IPs are their edge nodes. Amazon returns more IPv6 addresses, because IPv6 is abundant and partially accepted by most networks today. Amazon also returns two hops of CNAME resolutions. Wikipedia has the least amount of returning results.

2. Resolve ox.ac.uk

```
ubuntu@ubuntu:~$ nslookup www.ox.ac.uk
Server:      127.0.0.53
Address: 127.0.0.53#53

Non-authoritative answer:
www.ox.ac.uk canonical name = www.ox.ac.uk.cdn.cloudflare.net.
Name:   www.ox.ac.uk.cdn.cloudflare.net
Address: 104.20.34.13
Name:   www.ox.ac.uk.cdn.cloudflare.net
Address: 172.66.169.161
```

www.ox.ac.uk is a CNAME (alias) pointing to `www.ox.ac.uk.cdn.cloudflare.net`, a Cloudflare CDN domain. It resolves to 2 IPv4 addresses: 104.20.34.13 and 172.66.169.161, where both are the edge nodes from cloudflare. Using a CDN provides DDoS protection, caching, and global load balancing.

3. Resolve ox.ac.uk with -norec

```
ubuntu@ubuntu:~$ nslookup -norec www.ox.ac.uk
Server:      127.0.0.53
Address: 127.0.0.53#53

** server can't find www.ox.ac.uk: REFUSED

ubuntu@ubuntu:~$ nslookup -debug -norec www.ox.ac.uk
Server:      127.0.0.53
Address: 127.0.0.53#53
```

QUESTIONS:

www.ox.ac.uk, type = A, class = IN

ANSWERS:

AUTHORITY RECORDS:

ADDITIONAL RECORDS:

** server can't find www.ox.ac.uk: REFUSED

With `-norec`, the DNS server does not perform recursive resolution. Our local resolver (127.0.0.53) is a caching resolver that is not authoritative for the `ox.ac.uk` zone. It does not have the answer cached, so it refuses the query because it cannot answer without recursing. The `-debug` output confirms that no ANSWERS, AUTHORITY, or ADDITIONAL records were returned.

4. Different DNS servers with `-norec`

```
ubuntu@ubuntu:~$ nslookup -norec www.ox.ac.uk 208.67.222.222
;; Got SERVFAIL reply from 208.67.222.222
Server:      208.67.222.222
Address: 208.67.222.222#53
```

** server can't find www.ox.ac.uk: SERVFAIL

```
ubuntu@ubuntu:~$ nslookup -norec www.ox.ac.uk 8.8.8.8
;; Got SERVFAIL reply from 8.8.8.8
Server:      8.8.8.8
Address: 8.8.8.8#53
```

** server can't find www.ox.ac.uk: SERVFAIL

```
ubuntu@ubuntu:~$ nslookup www.ox.ac.uk 208.67.222.222
Server:      208.67.222.222
Address: 208.67.222.222#53
```

Non-authoritative answer:

www.ox.ac.uk canonical name = www.ox.ac.uk.cdn.cloudflare.net.

Name: www.ox.ac.uk.cdn.cloudflare.net

Address: 172.66.169.161

Name: www.ox.ac.uk.cdn.cloudflare.net

Address: 104.20.34.13

```
ubuntu@ubuntu:~$ nslookup www.ox.ac.uk 8.8.8.8
Server:      8.8.8.8
Address: 8.8.8.8#53
```

```
Non-authoritative answer:
www.ox.ac.uk canonical name = www.ox.ac.uk.cdn.cloudflare.net.
Name:      www.ox.ac.uk.cdn.cloudflare.net
Address: 104.20.34.13
Name:      www.ox.ac.uk.cdn.cloudflare.net
Address: 172.66.169.161
```

DNS Server	With recursion	Without recursion
Local (127.0.0.53)	Success (2 IPs)	REFUSED
OpenDNS (208.67.222.222)	Success (2 IPs)	SERVFAIL
Google (8.8.8.8)	Success (2 IPs)	SERVFAIL

Without recursion, all three DNS servers fail to respond the query. Public recursive resolvers (OpenDNS, Google) are not authoritative for the ox.ac.uk zone. They only serve cached data or perform recursion per user's instruction. When recursion is disabled and they don't have a cached answer, they fail with SERVFAIL. The local resolver returns REFUSED instead because it's configured as a stub resolver that always needs to forward.

With recursion enabled, all servers successfully traverse the DNS hierarchy (root → .uk TLD → ox.ac.uk authoritative → Cloudflare) to resolve the name.

5. Resolve a non-existent address

```
ubuntu@ubuntu:~$ nslookup nonononononononononononononononononono.com
Server:      127.0.0.53
Address: 127.0.0.53#53

** server can't find nonononononononononononononononononono.com: NXDOMAIN
```

NXDOMAIN (Non-Existent Domain) is the standard DNS response code indicating that the queried domain name does not exist in the DNS hierarchy. The recursive resolver traversed the root and .com TLD servers, which confirmed no such domain is registered.

6. Reverse lookup

```
ubuntu@ubuntu:~$ nslookup 8.8.8.8
8.8.8.8.in-addr.arpa name = dns.google.

ubuntu@ubuntu:~$ nslookup 142.251.153.119
** server can't find 119.153.251.142.in-addr.arpa: NXDOMAIN

ubuntu@ubuntu:~$ nslookup 104.20.34.13
```

```
** server can't find 13.34.20.104.in-addr.arpa: NXDOMAIN
```

```
ubuntu@ubuntu:~$ nslookup 127.0.0.1  
1.0.0.127.in-addr.arpa name = localhost.
```

```
ubuntu@ubuntu:~$ nslookup 192.168.1.1  
** server can't find 1.1.168.192.in-addr.arpa: NXDOMAIN
```

IP Address	Reverse Lookup Result	Reason
8.8.8.8	dns.google	Google explicitly configures PTR records for their public DNS service
127.0.0.1	localhost	Loopback address has a well-known PTR record in the DNS
142.251.153.119 (Google)	NXDOMAIN	Many public IPs (especially front-end/CDN IPs) lack PTR records for privacy/operational reasons
104.20.34.13 (Cloudflare)	NXDOMAIN	Same reason as above. CDN edge IPs typically do not have rDNS record
192.168.1.1 (private)	NXDOMAIN	Private IP ranges are not available in public DNS

PTR records are optional and must be manually configured by the IP address owner via their ISP or RIR. Public-facing infrastructure like `dns.google` has them for operational transparency, while most CDN/proxy IPs and private addresses do not.

DYN DNS Failure

1. Where were Dyn DNS servers located?

Dyn had 20 data centres worldwide. From their network map:

North America: Chicago, Los Angeles, Seattle, Washington DC, Miami, Dallas, Palo Alto, Newark

Europe: London, Warsaw, Frankfurt, Amsterdam

Asia Pacific: Tokyo, Hong Kong, Singapore, Sydney, Mumbai

South America: São Paulo

China: Beijing, Shanghai

2. Why did the DNS fail, and what was the nature of the failure?

Three waves of DDoS attacks hit Dyn on October 21, 2016. The source was the **Mirai botnet** from IoT devices infected with malware. These devices sent tens of millions of DNS lookup requests to Dyn's servers.

The attack had two components:

- **TCP SYN floods** against port 53 of Dyn's DNS servers
- **Subdomain (prepend) attacks** which is random subdomain queries that forced expensive lookups, burning CPU resources

3. Which companies were affected by the failure, and how?

- **Twitter:** Users across Europe and North America couldn't access the platform for hours because DNS queries for twitter.com failed
- **Netflix:** Streaming became unavailable for many users because netflix.com DNS could not resolve
- **Amazon:** Only experienced slowdowns, not a full outage. They also used UltraDNS as a second DNS provider, so they had a fallback

Other affected companies: Spotify, Reddit, GitHub, PayPal, Airbnb, Etsy, Shopify, Pinterest, SoundCloud, PlayStation Network, HBO, CNN, The New York Times, and many others.

4. Users in how many countries experienced the effect of the DNS failure?

Users in Europe and North America were most affected, especially along the US East Coast. During the second wave the impact went global, affecting all geographies where Dyn had data centres. Dozens of countries were impacted including the US, Canada, UK, France, Germany, Sweden, and others across Western Europe.

5. How were different government services affected? In particular, consider the effect on Sweden.

Sweden was hit because the **Swedish Civil Contingencies Agency (MSB)** and the **Swedish Government website (regeringen.se)** were both Dyn customers. Both sites became inaccessible during the attack, which hurt the government's ability to communicate with citizens online.

In the US, the White House monitored the situation and the Department of Homeland Security launched an investigation. The US Election Assistance Commission warned that similar attacks could disrupt electronic voting for overseas military voters.

6. How was the problem resolved?

Dyn's NOC team worked with upstream ISPs and mitigation partners to filter the malicious traffic. The first wave was mitigated after about 2 hours with traffic filtering at the network edge. The second wave was resolved in just over an hour. The third wave was blocked at the perimeter with no customer impact. Full service was restored by around 6:11 PM ET.

After the attack, Dyn worked with law enforcement and security firms to analyse the source, confirming the Mirai botnet as the cause.

7. Comment about steps taken by companies and government agencies to mitigate the effects of future DNS failures.

Companies:

- **Use multiple DNS providers:** Amazon survived better because they had UltraDNS as backup. Single-provider customers went completely down
- **Use DDoS mitigation services:** Akamai, Cloudflare, AWS Shield can absorb large attacks
- **Monitor DNS proactively:** ThousandEyes and similar tools give early warnings
- **Set shorter DNS TTLs:** Faster failover during outages
- **Secure IoT devices:** Change default passwords, keep firmware updated, segment IoT on separate networks

Government agencies:

- **DNS resilience plans:** Sweden's MSB now includes DNS in its cyber security strategy
- **Use multiple vendors:** Government sites should not depend on a single DNS provider
- **Share threat intelligence:** Collaboration between governments, ISPs and researchers helps detect botnets earlier
- **IoT security regulations:** After this attack, the UK (PSTI Act) and California (SB-327) introduced IoT security standards requiring minimum security baselines for connected devices

DNS Failure Case Study — Cloudflare 1.1.1.1 (July 14, 2025)

What happened and where?

On July 14, 2025, Cloudflare's public DNS resolver 1.1.1.1 became globally unavailable for 62 minutes (21:52–22:54 UTC). Users who had 1.1.1.1 configured as their DNS server could not resolve any domain names. Cloudflare operates a global anycast network with hundreds of edge servers worldwide, so this was a complete global outage.

Why did the DNS fail?

The failure was due to an internal configuration error. On June 6, 2025, a Cloudflare engineer made a configuration change to prepare a service topology for a new Data Localization Suite (DLS) service that was not yet in production. The change accidentally included the 1.1.1.1 Resolver's IP prefixes alongside the intended prefixes. This error hid for over a month because the DLS service was not live.

On July 14, a second change was made to the same DLS service: an offline test data centre location was added. This triggered a global refresh of the associated routes. Because the 1.1.1.1 prefixes were linked to the DLS service from the June 6 error, the 1.1.1.1 Resolver's service topology was reduced from all locations down to a single offline location. All 1.1.1.1 BGP prefixes were immediately withdrawn from the global routing table worldwide.

The prefix 1.1.1.0/24 had historical usage before Cloudflare acquired it in 2018, so when Cloudflare's routes disappeared, dormant routing entries from Tata Communications (AS4755) became visible, which looked like a BGP hijack but was not the cause of the outage.

Who was affected and how?

Anyone using Cloudflare's 1.1.1.1, 1.0.0.1, or their IPv6 equivalents (2606:4700:4700::1111, 2606:4700:4700::1001) as their DNS resolver. This included:

- **Individuals** who manually set 1.1.1.1 on their devices or routers
- **Organisations** using Cloudflare Gateway DNS for content filtering and security
- **WARP users** who rely on 1.1.1.1 for DNS resolution

DoH (DNS-over-HTTPS) users were mostly unaffected because they connect via the domain `cloudflare-dns.com` which uses different IP addresses.

Could it have been prevented?

Yes. The root cause was a legacy configuration system that:

1. Used hard-coded lists of data centre locations tied to IP prefixes. Error-prone and hard to audit
2. Didn't use progressive/staged deployment. Changes propagated to every data centre instantly with no staging testing
3. Required maintaining two systems (legacy + modern) that needed to stay in sync

If the change had gone through staged canary deployments with health monitoring, the error would have been caught before reaching all global data centres.

How did Cloudflare react and recover?

Time (UTC)	Event
21:48	Configuration change triggers global route withdrawal
21:52	DNS traffic to 1.1.1.1 begins dropping globally
22:01	Internal alerts fire: incident declared
22:20	Configuration reverted and BGP prefixes begin being re-advertised
22:54	Full restoration. Traffic returns to normal levels

Within 19 minutes of detection, Cloudflare identified the issue and deployed a revert. The revert restored BGP announcements instantly, but it took another 34 minutes for all edge servers to rebind the IP addresses because Cloudflare's change management system applies progressive rollouts by default for safety. They accelerated this process given the severity.

What steps have been taken to prevent future failures?

From Cloudflare's post-mortem:

- **Deprecate legacy addressing systems:** The old system that required hard-coding IP-to-location mappings will be replaced entirely with the modern approach that uses progressive, health-monitored deployment
- **Stage all addressing deployments:** Every topology change will go through canary deployments with automated health checks before rolling out globally
- **Improve test coverage:** Better automated testing for service topology configuration changes

I would also propose:

- **Automated validation of dormant configs:** Periodic jobs that detect when a pre-production configuration accidentally references production IP space and flag it before it can cause harm
- **Cross-team review for topology changes:** Any change touching the global route advertisement system should require sign-off from both the DLS team and the core DNS team